

BattleOps: Week 3

Theme: Kubernetes Security, Platform Hardening & Real-World Incident Prevention

Objective: Apply Kubernetes security concepts in real production-like situations, debug misconfigurations, enforce policies, and secure workloads end-to-end.

MONDAY: Enforcing Runtime Security (PSA, Seccomp, AppArmor)

ADVANCED CONCEPTS

- How attackers escalate privilege inside a cluster
- Why “privileged containers” are production risks
- How PSA actually blocks unsafe workloads
- Understanding runtime syscall filtering (seccomp)
- Blocking malicious containers using AppArmor

KEY OUTCOMES

- You can enforce cluster-wide restricted PSA
 - You can diagnose WHY a pod was blocked
 - You can identify dangerous securityContext fields
 - You can apply a seccomp profile & verify enforcement
-

WHAT TO PRACTICE

Lab 1: Break PSA intentionally

Deploy a malicious-style pod:

securityContext:

privileged: true

allowPrivilegeEscalation: true

Observe: `kubectl describe pod <name> || kubectl get events`

Lab 2: Fix & Enforce Restricted Mode

Patch pod → make it compliant.

Apply namespace PSA = restricted.

Lab 3: Apply a Seccomp Profile

Use the default `/runtime/default`, then apply a custom local profile.



WHERE TO STUDY

- PSA Enforcement examples (k8s docs)
 - Seccomp deep dive; Sysdig blog
 - AppArmor profiles in Kubernetes
-

INFRATHRONE BATTLEOPS NOTES

- Privilege escalation is ALWAYS the first attempt of an attacker.
 - PSA is not optional; it's mandatory for multi-tenant clusters.
 - Seccomp reduces kernel attack surface by 80%.
-